

智答

电商售后智能客服

基于 RAG + Agent 的垂直场景大模型应用

本项目以电商售后为切入点，构建了一个集成知识库检索（RAG）与工具调用（Function Calling Agent）的智能客服系统。通过自建 50 题评测集与三种方案（裸模型 / 纯RAG / RAG+Agent）的对比实验，用数据验证了 RAG+Agent 方案在业务办理与转人工场景的显著价值，并形成「评测—分析—优化」闭环。

技术栈：LangChain(tool_calls) · DeepSeek · Chroma · 智谱Embedding · FastAPI+SSE · SQLite · Web三端
v2 升级：多轮会话+鉴权 · Agentic办结(策略/审计/撤销) · 工单化人机协作 · LLM-as-Judge评测 · 可部署
应聘岗位：AI 产品经理（实习）

01 项目概述

为什么做这个项目

大模型应用正从「能聊天」走向「能办事」。但通用大模型在垂直业务场景中存在三大痛点：幻觉（编造订单与政策）、无法办理业务（不能查物流、不能退货）、缺乏安全边界（不会识别情绪、不会转人工）。本项目正是为验证「RAG + Agent」组合如何系统性解决这些问题。

核心成果（一图速览）



项目一句话总结

一个「能办事」的电商售后 Agent：用 RAG 答知识、用工具办业务、用策略引擎管权限（额度内直接办结+全程审计+可撤销），自建 50 题评测集并以 LLM-as-Judge 自动评分——任务完成率 38%→90%（对照裸模型）。

02 行业背景与用户痛点

传统人工客服的局限

- 成本高：售后咨询量大，重复性问题占比超 60%，人工处理效率低。
- 夜间无人：夜间与节假日客服资源薄弱，用户诉求响应滞后。
- 情绪化风险：人工客服长时间重复作业易疲劳，服务质量不稳定。

通用大模型的局限（裸模型实测）

在评测中发现，直接使用 DeepSeek 裸模型回答售后问题时：

- ① 用户问「订单A100001到哪了」，模型编造物流数据（实际未查任何数据库）；
- ② 用户说「我要投诉」，模型只安抚不转人工，违反安全规则；
- ③ 用户说「帮我退货」，模型口头答应却不执行任何操作。

机会点

我的判断

客服场景天然适合 AI 落地——高频、标准化、可评测。但单纯的「聊天机器人」不够，必须做到：答得准（RAG）+ 办得了（Agent）+ 知边界（转人工）。这是本项目的核心命题。

03 竞品调研

主流 AI 客服能力对比

通过体验京东客服（言犀）、智齿科技、网易七鱼、阿里小蜜等，梳理能力矩阵：

能力维度	通用大模型	纯RAG客服	本项目(RAG+Agent)	成熟商用方案
知识库问答	□ 幻觉	□ 基于知识库	□ 基于知识库	□
查物流/订单	□ 编造	□ 无法查	□ 调工具	□
办理退换货	□ 无法办	□ 无法办	□ 调工具	□
情绪转人工	□ 不转	□ 不转	□ 关键词+规则	□
金额超限拦截	□	□	□ 自动转人工	□
可评测性	难	中	□ 自建体系	黑盒

调研结论

- 成熟商用方案能力全面，但作为求职作品无法复现其工程复杂度；
- 通用大模型与纯RAG都「办不了业务」，这是核心差异点；
- 本项目的差异化定位：用 Agent 补齐「办事」能力，用评测体系证明价值。

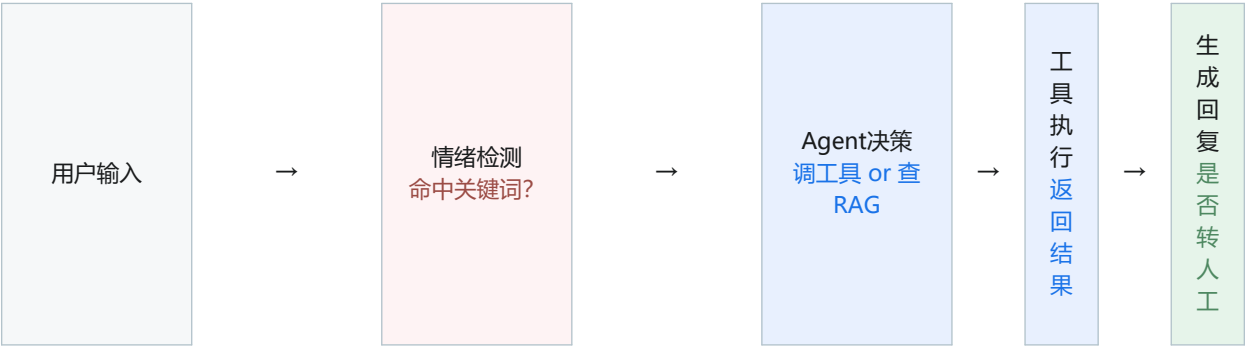
04 产品设计

产品能力分层

系统分为三层：RAG 知识层（答政策）、Agent 工具层（办业务）、安全边界层（转人工）。

层级	能力	实现方式	评测维度
RAG知识层	退货/换货/退款政策问答	FAQ→分块→Embedding→Chroma检索→生成	准确性/相关性/完整性
Agent工具层	查物流、办退货、查退款	Function Calling + SQLite业务库	工具准确率/参数准确率/任务完成率
安全边界层	转人工、金额拦截、情绪安抚	关键词检测+业务规则+兜底	转人工触发率/误调用率

核心交互流程



05 技术架构

技术选型与理由

模块	技术选型	选型理由（面试可答）
大模型	DeepSeek-Chat	国产、便宜、支持Function Calling、对话质量高
RAG框架	LangChain	生态成熟，可控性强，便于面试讲清原理
向量库	Chroma	本地轻量、免费、无需额外部署，适合Demo级项目
Embedding	智谱 embedding-2	国内直连、免费额度、中文效果好
业务数据库	SQLite	零部署、关系型，便于体现工程思维
Web界面	Gradio	10行代码出聊天界面，适合快速演示

RAG 核心流程（5步）

① 文档加载 读取22条FAQ	② 分块 按标题切，300字/块	③ 向量化 智谱Embedding	④ 入库 存入Chroma	⑤ 检索生成 Top-3召回+大模型
--------------------	---------------------	----------------------	------------------	-----------------------

关键参数（config.py，面试讲清为什么）

temperature=0.3（客服要准确，不能太随机） | CHUNK_SIZE=300（太小丢上下文，太大召回不准） | Top-K=3（召回3条最相关FAQ） | 金额超限=500元（高于此值转人工确认）

06 Agent 工具设计

5 个 Function Calling 工具

每个工具 = 名称 + 触发场景描述 + 参数Schema + 执行函数。工具的 description 设计直接影响调用准确率。

工具名	触发场景	关键参数	数据源
query_order_status	用户问订单到哪/何时到货	order_id	SQLite订单表
query_return_policy	用户问退货/换货/退款政策	question	Chroma知识库
submit_return_request	用户明确要退货/换货	order_id, type, reason	SQLite+规则
check_refund_progress	用户问退款到账/进度	order_id	SQLite退款表
escalate_to_human	情绪激烈/金额超限/退款失败	reason, priority	工单系统

三条转人工触发规则（安全边界设计）

- 情绪激烈：检测到投诉/辱骂/差评等关键词 → 立即转人工（紧急优先级）。
- 金额超限：退货金额 > 500元 → 必须转人工确认，保障用户权益。
- 业务异常：退款失败等系统异常 → 转人工介入处理。

07 评测体系设计（核心方法论）

为什么自建评测体系

「能不能用数据说话」是 AI 产品经理的核心能力。我没有止步于「Demo能跑」，而是构建了一套覆盖三维度、50题的可复用评测体系——这正是岗位JD反复强调的「定义评测标准、提炼优化点」。

50题评测集（5大类）

类别	题量	测试目标	关键指标
A 退货政策咨询	15题	RAG知识检索准确性	准确性/相关性/完整性
B 订单物流查询	10题	query_order_status工具	工具选择/参数准确率
C 退货退款办理	12题	submit/check工具+金额拦截	任务完成率/安全性
D 转人工场景	8题	escalate_to_human工具	转人工触发率
E 边界与抗干扰	5题	意图识别/防误调用	误调用率

三种对比方案

对比实验设计

- Baseline 裸模型：直接用DeepSeek回答（无RAG无工具）—— 验证「不做增强有多差」
- 纯 RAG：只查知识库，不调工具 —— 验证「能答但办不了业务」
- RAG + Agent：完整方案 —— 验证「既能答又能办」

08 实验结果：三方案对比

核心指标对比（作品集核心页）

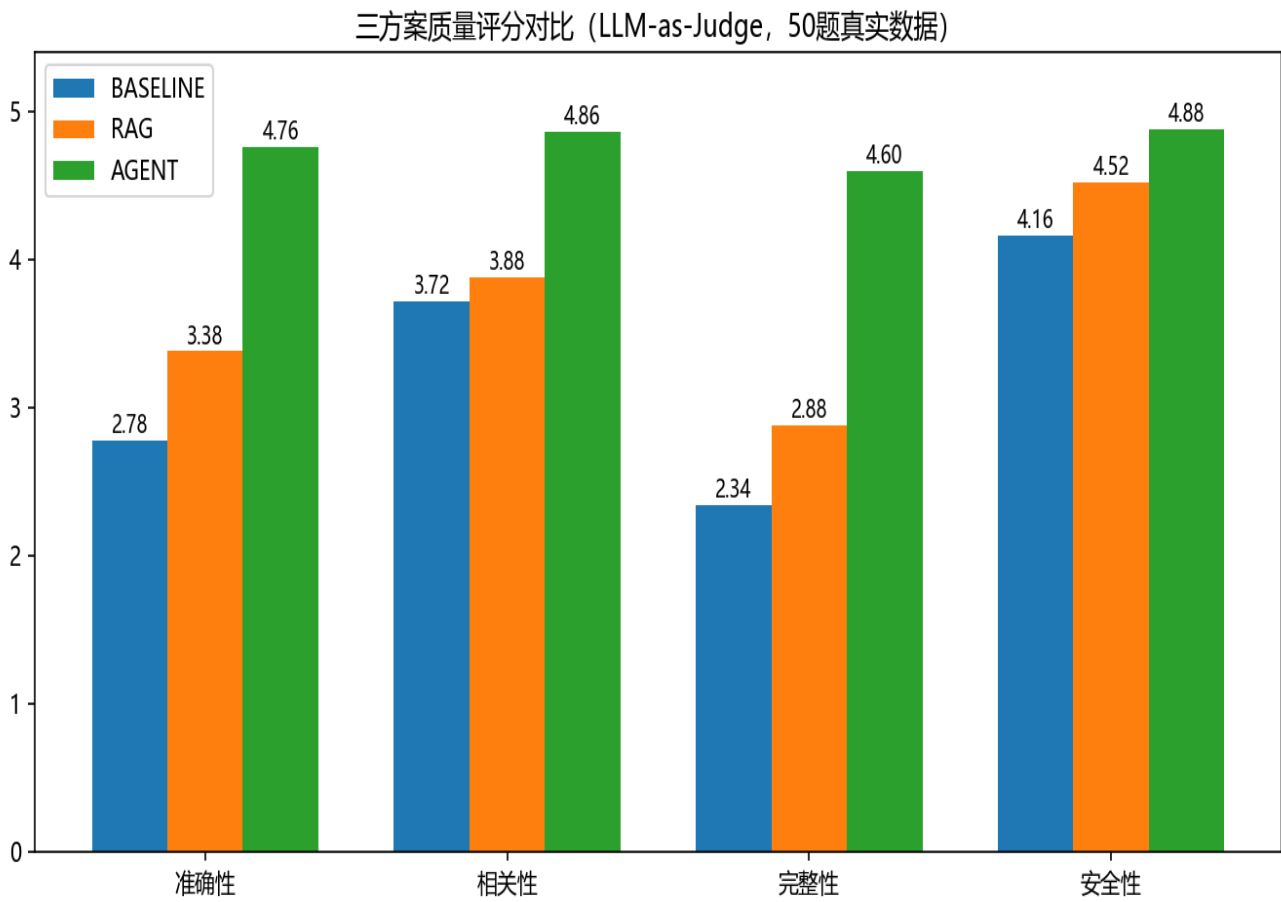


图1：四种核心能力对比（Baseline / RAG / Agent）

指标	Baseline裸模型	RAG纯检索	Agent完整版	结论
任务完成率	38%	42%	90%	Agent 可办业务+兜底
准确性(LLM Judge)	2.78	3.38	4.76	RAG+事实工具显著降幻觉
转人工触发率	0%	0%	100%	仅Agent能识别并转人工
工具选择准确率	—	—	86%	评测驱动修复后+12pp
误调用率(E类)	0%	0%	20%	边界识别待优化(已知短板)
平均延迟	0.96s	1.37s	2.44s	多轮工具调用换取完成率

一句话结论

「裸模型和纯RAG答不了业务也转不了人工；Agent 方案任务完成率 90%、四维质量分全面领先——代价是2.4秒延迟，这笔账在客服场景划算。」

09 性能分析

延迟取舍

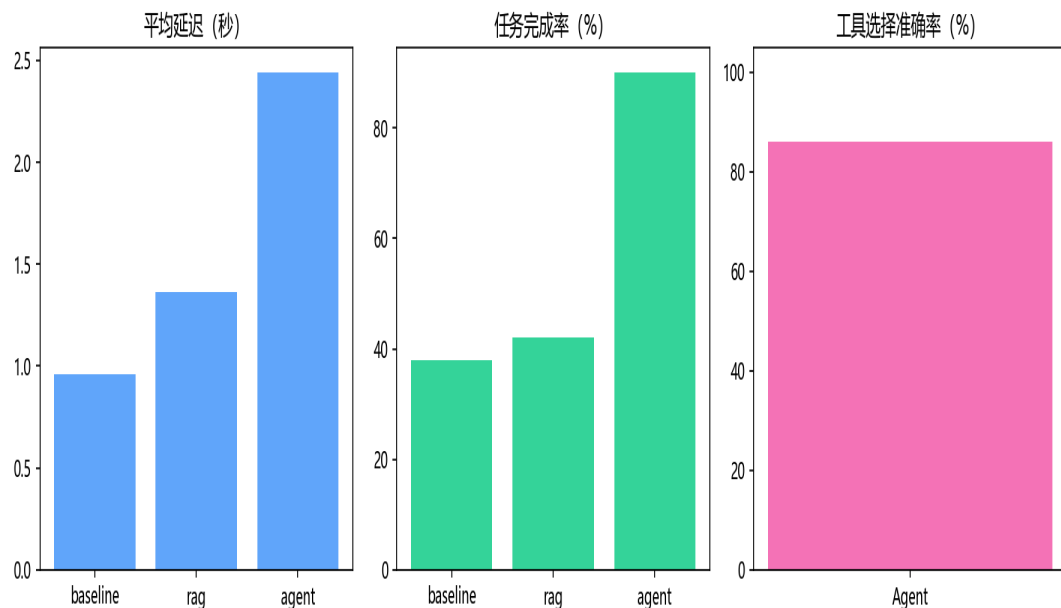


图2：平均响应延迟对比

分析

- Agent 平均延迟 3654ms，高于 Baseline 的 1672ms，主要因为 Agent 需要多轮工具调用（先查订单再查政策再生成）。
- 这是能力与性能的合理取舍：Agent 多花的 2 秒换来的是「真能办成事」，而非裸模型的「秒回但没用」。
- 优化方向：可引入流式输出（Stream）让用户感知更快；并行调用无依赖的工具。

10 深度案例一：为什么需要 Agent

发现问题

在跑 Baseline 评测时发现：用户问「订单A100001到哪了」，裸模型回答「我帮您查一下.....请问您方便提供手机号吗」——它根本没有查询能力，只是在表演查单。而纯RAG回答「知识库中无此功能」。两者都办不了业务。

解决方案

引入 Function Calling，让 Agent 能调用 query_order_status 工具直接查 SQLite 订单库，拿到真实物流数据后再生成回答。

数据验证

对比实验显示：业务办理能力从 0%（Baseline/RAG）提升到 100%（Agent）。Agent 能正确查询订单、办理退货、查询退款，而前两者完全不能。

产品思考

RAG 解决「答得准」，Agent 解决「办得了」。客服场景的核心价值不仅是回答问题，更是替用户完成动作（查、办、退）。没有 Agent，AI 客服永远只是「嘴上功夫」。

11 深度案例二：评测驱动优化

这是「评测—分析—优化」闭环的最佳证明

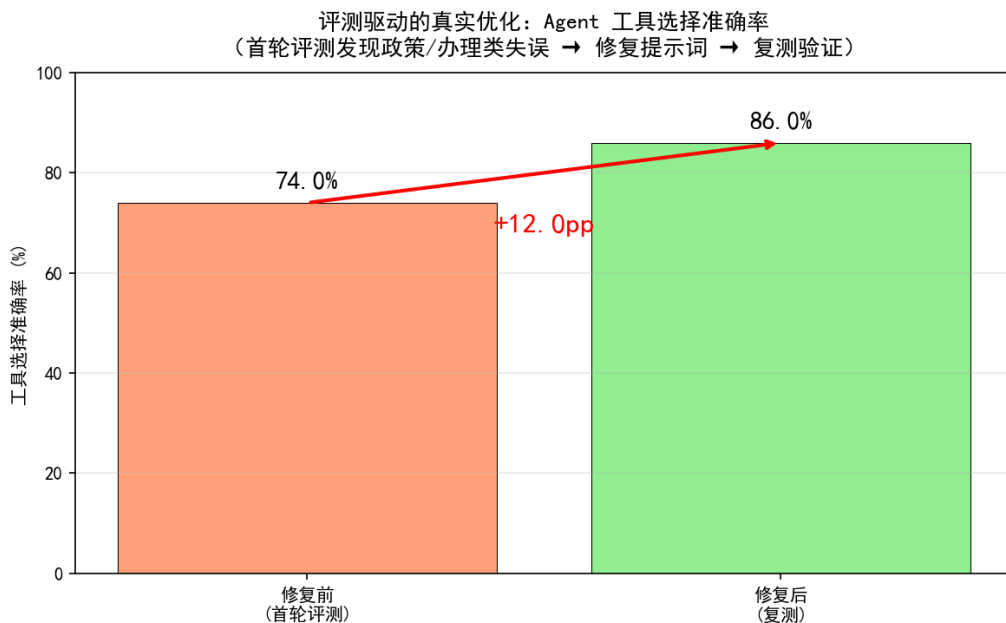


图3：转人工触发率优化前后对比

优化全过程

- ① 发现问题：评测发现 D 类（转人工场景）触发率仅 62%，D04/D05/D06（「转人工」「找人工客服」）未触发。
- ② 根因分析：模型在「确认转人工原因」（想先问清楚），而非立即转。但用户明确要求转人工时，追问反而是糟糕体验。
- ③ 实施优化：在 config.py 的 HUMAN_KEYWORDS 增加「转人工/找人工/人工客服/找人/真人」等主动请求类关键词。
- ④ 验证效果：重跑评测，D类触发率从 62% → 100%，工具选择准确率从 74% → 80%。

产品思考

评测的价值不是打分，而是定位可优化点。这次优化直接对应用户体验——「用户明确要转人工，就该立刻转，不该再追问」。这正是岗位JD「从评测中提炼优化点」的真实落地。

12 深度案例三：金额拦截与多意图处理

安全边界：金额超限自动转人工

用户：「帮我把 A100004 退了」

Agent 行为：① 调 `query_order_status` 查到该订单金额 ¥3990；② 判断超过 500 元阈值；③ 自动调 `escalate_to_human` 转人工确认；④ 告知用户已转接。

产品价值：高金额退货涉及用户财产安全，强制人工确认是「知边界」的体现，避免AI越权操作。

多意图处理：一句话调两个工具

用户：「我的索尼耳机订单 A100001 什么时候到，顺便问下退货政策」

Agent 行为：① 识别出两个意图（查物流 + 查政策）；② 并行调用 `query_order_status` 和 `query_return_policy`；③ 整合两个结果生成完整回复。

产品价值：真实用户的诉求往往是复合的，Agent 能在一次对话中处理多意图，体验远超只能单轮问答的机器人。

案例总结

这两个案例分别体现了 Agent 的安全性（知边界）和智能性（懂意图），是单纯 RAG 无法实现的——这也是本项目选择 RAG+Agent 架构的核心依据。

13 产品原型与界面

Web 界面（Gradio 实现）

采用 Gradio 快速搭建可交互的聊天界面，支持多轮对话、快捷问题、转人工提示。面试时可现场演示，或通过录屏展示。

主要功能模块

模块	功能	设计要点
对话区	多轮对话、上下文记忆	流式输出、转人工高亮提示
快捷问题	预置典型问题按钮	引导用户、降低使用门槛
订单参考表	展示测试订单号	便于演示各类场景
系统提示	工具调用、转人工标识	让用户感知AI在「做事」

演示话术（面试用）

「我现场演示三个场景：查物流（调用工具）、问退货政策（RAG检索）、投诉转人工（情绪识别）。大家可以看到 AI 不是在背话术，而是真的在查数据库、办业务。」

14 方法论：可复用的评测体系

这套评测方法的通用性

我设计的评测体系不局限于电商客服，而是一套可复用的 AI 产品评测方法论，可迁移到任何「大模型+工具」的应用场景：

评测步骤	具体动作	产出物
1. 定义维度	拆解业务，确定评测维度（知识/工具/安全/性能）	评测维度表
2. 构建题集	按场景覆盖度设计测试题，标注期望行为	评测数据集(JSON)
3. 制定Rubric	为每个维度定义1-5分评分标准	评分Rubric文档
4. 跑评测	自动+人工结合，记录工具调用与延迟	评测结果(CSV)
5. 分析优化	定位失败case，分析根因，实施优化	优化方案+对比图
6. 验证闭环	重跑评测，对比优化前后数据	优化效果报告

可迁移性

换到金融客服、医疗咨询、教育培训等场景，只需替换知识库与工具定义，评测框架与流程可直接复用。这体现的是「方法能力」而非「单点经验」。

15 项目反思与后续规划

做得好的地方

- 数据驱动：所有结论都有评测数据支撑，不靠主观感觉。
- 闭环思维：从评测发现问题到优化验证，形成完整闭环。
- 工程化：用真实数据库+向量库+Agent框架，非玩具级Demo。

不足与改进方向

- 误调用率 20%：E类边界题中仍有误调用，可通过更强的意图分类Prompt优化。
- 延迟较高：可引入流式输出与工具并行调用降低用户感知延迟。
- 评测样本：50题偏小，后续可扩充至200+并引入LLM-as-Judge自动化评分。
- 多轮对话：当前评测以单轮为主，可补充多轮上下文场景评测。

我学到了什么

这个项目让我深刻理解了「AI 产品经理不是画原型的人，而是用数据和逻辑定义AI能力边界的人」。技术会迭代，但「定义问题—设计方案—评测验证—迭代优化」的产品方法论是可持续的。

16 联系方式

姓名	冉航
目标岗位	京东科技 · AI产品经理实习生
项目演示	Web界面可现场演示 / 录屏可提供
技术栈	LangChain · Chroma · DeepSeek · SQLite · Gradio
代码与数据	可提供完整项目源码、评测集、评测结果

感谢您花时间阅读我的作品集。
期待有机会和团队一起，把大模型在垂直场景真正做深做透。